

PEMROGRAMAN C++ 20 BOLA

Nama: Ahmad Faiz Abdillah

NIM: 26032014016

Kelas: 2026C

Prodi: Kecerdasan Artifisial

Persamaan Matematis yang Mendasari

1. Dekomposisi Kecepatan dari Sudut (Vektor Polar $\rightarrow$ Kartesian)

Kecepatan awal dibangun dari sudut acak θ dan kelajuan v :

$$v_x = v \cos \theta, v_y = v \sin \theta$$

$$v_x = v \cos \theta, v_y = v \sin \theta$$

di mana $\theta \sim U(0, 2\pi)$ dan $v \sim U(0.5, 2.0)$. Ini adalah **konversi koordinat polar ke Kartesian**, memastikan arah gerak benar-benar acak ke segala arah dengan kelajuan bervariasi.

2. Pembaruan Posisi (Sama seperti sebelumnya)

$$x_{n+1} = x_n + v_x, y_{n+1} = y_n + v_y$$

$$x_{n+1} = x_n + v_x, y_{n+1} = y_n + v_y$$

3. Pantulan Dinding dengan Clamping

Berbeda dari versi sebelumnya, di sini posisi juga **dijepit (clamped)** ke batas agar bola tidak "menembus" dinding karena floating point:

$$x = \begin{cases} 1 & \text{jika } x \leq 1 \\ lebar - 2 & \text{jika } x \geq lebar - 2 \\ x & \text{lainnya} \end{cases}$$
$$x = \begin{cases} 1 & \text{jika } x \leq 1 \\ lebar - 2 & \text{jika } x \geq lebar - 2 \\ x & \text{lainnya} \end{cases}$$

dan kecepatan dipaksa mengarah menjauhi dinding menggunakan nilai absolut:

$$v'_x = |v_x| \text{ (saat di dinding kiri), } v'_x = -|v_x| \text{ (saat di dinding kanan)}$$
$$v'_x = |v_x| \text{ (saat di dinding kiri), } v'_x = -|v_x| \text{ (saat di dinding kanan)}$$

4. Deteksi Tabrakan Antar Bola (Jarak Euclidean)

Dua bola dianggap bertabrakan jika jarak pusatnya kurang dari jumlah radius:

$$d = \sqrt{(x_b - x_a)^2 + (y_b - y_a)^2}$$

$$d = \sqrt{(x_b - x_a)^2 + (y_b - y_a)^2}$$

Tabrakan terjadi jika $d < 2R$ Tabrakan terjadi jika $d < 2R$

dengan $R = \text{radius bola}$ (di sini $R = 1$).

5. Resolusi Penetrasi (Position Correction)

Jika dua bola saling menembus ($d < 2R$), posisinya didorong menjauh sepanjang **vektor normal** $\hat{n}$:

$$\hat{n} = \left(\frac{x_b - x_a}{d}, \frac{y_b - y_a}{d} \right)$$

$$\mathbf{n} = (x_b - x_a, y_b - y_a)$$

$$\text{overlap} = \frac{2R - d}{2}$$

$$\vec{r}'_a = \vec{r}_a - \text{overlap} \cdot \hat{n}, \quad \vec{r}'_b = \vec{r}_b + \text{overlap} \cdot \hat{n}$$

Masing-masing bola didorong setengah dari total overlap, sehingga keduanya berkontribusi sama menyelesaikan penetrasi.

6. Kecepatan Relatif Sepanjang Normal

$$\vec{v}_{rel} = \vec{v}_a - \vec{v}_b$$

$$v_{rel} = v_a - v_b$$

$$v_n = \vec{v}_{rel} \cdot \hat{n} = (v_{a,x} - v_{b,x})n_x + (v_{a,y} - v_{b,y})n_y$$

Jika $v_n < 0$, artinya kedua bola sedang **menjauh**, sehingga tidak perlu diberi impuls (mencegah pantulan berulang/ganda).

7. Tumbukan Elastis Massa Sama (1D Elastic Collision di Sepanjang Normal)

Untuk dua benda bermassa sama yang bertumbukan elastis sempurna, hukum kekekalan momentum dan energi kinetik memberi hasil klasik: **komponen kecepatan sepanjang garis normal saling tertukar**. Program menyederhanakannya sebagai:

$$\text{impulse} = v_n$$

$$\text{impulse} = v_n$$

$$\vec{v}'_a = \vec{v}_a - \text{impulse} \cdot \hat{n}, \quad \vec{v}'_b = \vec{v}_b + \text{impulse} \cdot \hat{n}$$

Ini berasal dari rumus umum tumbukan elastis 1D massa sama ($m_a = m_b$):

$$v'_a = v_a - \frac{2m_b}{m_a + m_b} \cdot v_n \cdot \hat{n} = v_a - v_n \hat{n}$$

$$v_a' = v_a - m_a + m_b \cdot v_n \cdot \hat{n} = v_a - v_n \hat{n}$$

$$v_b' = v_b + \frac{2m_a}{m_a + m_b} \cdot v_n \cdot \hat{n} = v_b + v_n \hat{n} \quad v_b' = v_b + m_a + m_b \cdot v_n \cdot \hat{n} = v_b + v_n \hat{n}$$

$$\text{karena } \frac{2m_b}{m_a + m_b} = 1 \text{ maka } m_a = m_b \text{ maka } m_a = m_b.$$

Catatan fisis: hanya komponen kecepatan **sepanjang garis penghubung pusat bola** (normal) yang berubah; komponen tegak lurus (tangensial) tidak tersentuh — sama seperti bola biliar bertumbukan.

8. Kompleksitas Komputasi Deteksi Tabrakan

Dengan $n = 20$ bola, jumlah pasangan yang dicek tiap frame:

$$\binom{n}{2} = \frac{n(n-1)}{2} = \frac{20 \times 19}{2} = 190 \text{ pasangan}$$

$$(2n) = 2n(n-1) = 2 \times 20 \times 19 = 190 \text{ pasangan}$$

Ini adalah algoritma **brute-force** $O(n^2)$ — cukup untuk 20 bola tapi tidak efisien untuk ribuan bola (butuh spatial partitioning seperti grid/quadtree).

9. Frame Rate

$$\text{FPS} = \frac{1}{\Delta t_{\text{delay}}} = \frac{1}{0.03 \text{ s}} \approx 33.3 \text{ FPS}$$

$$\text{FPS} = \frac{1}{\Delta t_{\text{delay}}} = \frac{1}{0.03 \text{ s}} \approx 33.3 \text{ FPS}$$

Struktur Program

1. Struct Bola

cpp

```
struct Bola {  
    float x, y;    // posisi  
    float vx, vy;  // kecepatan  
    char simbol;  
};
```

Menggantikan array terpisah (posX, posY, vx, vy) dengan satu struktur data per bola — lebih rapi dan mudah di-scale ke banyak bola.

2. Inisialisasi Posisi Tanpa Tumpang Tindih

cpp

```
do {  
    bola[i].x = 2 + rand() % (lebar - 4);  
    bola[i].y = 2 + rand() % (tinggi - 4);  
    // cek jarak ke semua bola sebelumnya  
} while (!valid);
```

Setiap bola baru diberi posisi acak, lalu dicek jaraknya ke semua bola yang sudah ada. Jika terlalu dekat ($< 2 \times \text{radius}$), posisi diulang (rejection sampling).

3. Kecepatan Awal Acak (Arah & Besar)

cpp

```
float sudut = ((float) rand() / RAND_MAX) * 2.0f * M_PI;  
float speed = 0.5f + ((float) rand() / RAND_MAX) * 1.5f;  
bola[i].vx = cos(sudut) * speed;  
bola[i].vy = sin(sudut) * speed;
```

Kecepatan dibangun dari **sudut** dan **kelajuan** acak, lalu diuraikan ke komponen x dan y menggunakan trigonometri — ini penting secara matematis (dijelaskan di bawah).

4. Loop Utama: Update → Deteksi Tabrakan → Gambar

Tiga tahap per frame:

1. **Update posisi + pantul dinding** (pantulDinding)
2. **Cek tabrakan tiap pasangan bola** — $\binom{20}{2} = 190(220)=190$ pasangan per frame (pantulAntarBola)
3. **Render** ke grid karakter